Experiment No.: 9

PART A

Mini Project Title: ExamChain – Blockchain-Based Exam Paper Leak Prevention System

1. Abstract

Question paper leaks have repeatedly led to cancelled examinations, re-tests and a loss of trust in examination bodies. In the traditional system the paper is printed or e-mailed days before the exam, passes through many hands, and there is no reliable record of who accessed it. This mini project presents ExamChain, a decentralized application (DApp) built on the Ethereum blockchain that prevents and detects paper leaks. The question paper is encrypted with AES-256-GCM inside the paper setter's browser, and only its SHA-256 hash and a Keccak-256 commitment to the encryption key are stored on-chain. A Solidity smart contract time-locks the key: it can be released only after the exam start time and only after two of three exam authorities approve. Every action (upload, download, approval, release, verification) is recorded as a blockchain event, forming an immutable audit trail. Exam centers verify that the decrypted paper is authentic by comparing hashes, and anyone who finds a suspected leaked copy can prove the leak on-chain. All 9 unit tests pass, and the complete workflow was demonstrated through a web dashboard.

2. Introduction

A blockchain is a distributed, append-only ledger in which blocks of transactions are linked using cryptographic hashes. Once data is written and confirmed, it cannot be modified without changing every following block, which makes the record tamper-evident. Ethereum extends this idea with smart contracts: programs stored on the blockchain that execute exactly as written when their conditions are met, without any trusted middleman.

Examination security is fundamentally a trust problem. Paper setters, printers, couriers, officials and exam centers must all be trusted, and a single dishonest person can leak the paper. Existing logs are kept in central databases that can be edited. ExamChain applies the two key properties of blockchain, immutability and trustless rule enforcement, to remove this blind trust.

Problem statement: Design a system in which (i) the question paper remains unreadable until the exam starts, (ii) no single person can unlock it, (iii) every access is permanently logged, (iv) centers can verify the paper is genuine, and (v) a leak can be proven publicly.

Objectives:

- Keep the question paper encrypted until the exact exam start time using a smart-contract time-lock.
- Require multi-authority (2-of-3) approval before the decryption key can be released.
- Maintain an immutable, timestamped audit trail of all actions using blockchain events.
- Allow exam centers to verify integrity and authenticity of the paper using SHA-256 hashes.
- Provide a public mechanism to cryptographically prove a leak before the exam.

3. Methodology

The system involves five roles, each represented by a separate Ethereum wallet: Admin (University Exam Controller), Paper Setter, Exam Authorities (three officials), Exam Centers, and the Public. The workflow is:

1. **Schedule:** The Admin registers roles and calls scheduleExam(subject, setter, unlockTime).
2. **Encrypt:** The setter selects the paper. The browser generates a random 256-bit key K and IV, computes paperHash = SHA-256(paper), encFile = IV || AES-GCM(K, paper), encHash = SHA-256(encFile) and commitment = Keccak-256(K).
3. **Commit:** uploadPaper(id, paperHash, encHash, commitment) stores only these values on-chain. The commitment cannot be changed later.
4. **Distribute:** The encrypted .enc file is shared off-chain in advance. Centers check its hash against encHash and call logDownload().
5. **Approve:** Authorities call approveRelease(). At least two approvals are required.
6. **Release (time-lock):** At exam time the setter calls releaseKey(id, K). The contract checks block.timestamp >= unlockTime, approvals >= 2 and Keccak-256(K) == commitment.
7. **Decrypt & verify:** Centers read K from the blockchain, decrypt the paper, re-hash it and record the result with recordVerification().
8. **Leak proof:** Anyone can submit SHA-256 of a suspected leaked file via reportLeak(). If it matches paperHash before release, the exam is marked Compromised.

The key release uses a commit-reveal scheme: committing to the hash of the key at upload time guarantees that the key revealed at exam time is exactly the one used for encryption, so a fake key or swapped paper is rejected by the contract.

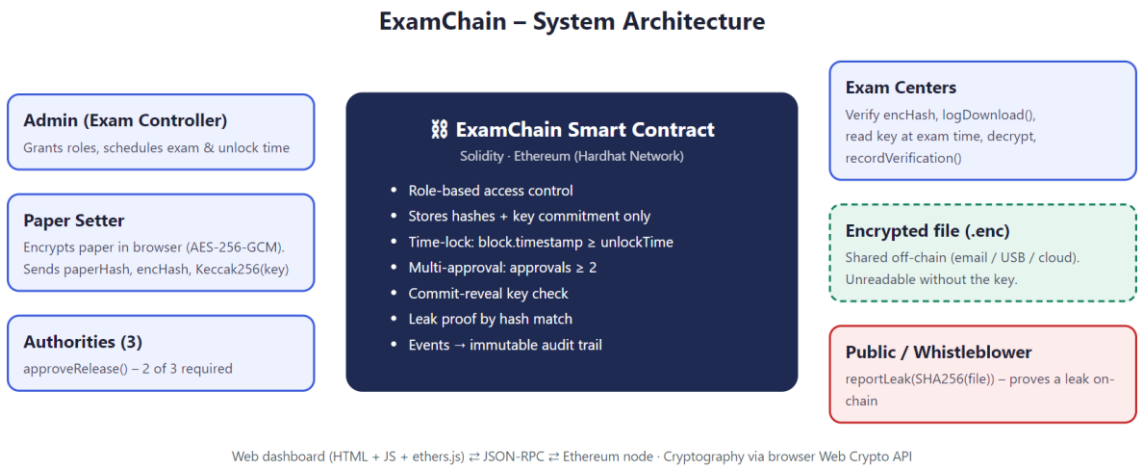


Figure 1: System architecture of ExamChain

4. Technology Used to Develop and Deploy the Application

Component	Technology	Purpose
Smart contract	Solidity 0.8.24	Business rules, time-lock, roles, audit events

Blockchain	Ethereum – Hardhat Network (chain ID 31337)	Local Ethereum node with 20 funded test accounts
Development framework	Hardhat 2.29, Node.js 20	Compile, test and deploy the contract
Testing	Mocha, Chai, Hardhat Network Helpers	Automated unit tests, time travel
Blockchain library	ethers.js v6	Frontend ↔ blockchain communication (JSON-RPC)
Frontend	HTML5, CSS3, JavaScript	Role-based web dashboard
Cryptography	Web Crypto API: AES-256-GCM, SHA-256; Keccak-256	Encryption, integrity, key commitment
Wallet (optional)	MetaMask	Signing transactions with a browser wallet
Web server	Node.js http module	Serves the dashboard at localhost:3000

Deployment: npm run node starts a local Ethereum node; npm run deploy compiles and deploys the contract, assigns demo roles to 8 accounts and writes the contract address and ABI to the frontend; npm start serves the dashboard. The same contract can be deployed unchanged to a public testnet (e.g. Sepolia) or a permissioned network.

5. Implementation Details

5.1 Smart contract (ExamChain.sol)

Each exam is stored as a struct containing the subject, setter, unlockTime, paperHash, encryptedHash, keyCommitment, revealedKey, fileName, approval count and status (Scheduled, Uploaded, Released, Compromised, Cancelled). Access is controlled with require() checks and modifiers.

Function	**Caller**	**Description**
addSetter / addAuthority / addCenter	Admin	Role-based access control
scheduleExam	Admin	Creates an exam with a future unlock time
uploadPaper	Assigned setter	Stores hashes and key commitment (only once, before exam)
logDownload	Exam center	Logs receipt of the encrypted paper
approveRelease	Authority	One approval per authority; 2 of 3 required
releaseKey	Assigned setter	Time-locked, approval-gated, commitment-checked key release
recordVerification	Exam center	Records whether the decrypted paper's hash matches
reportLeak	Anyone	Marks exam Compromised if suspected file hash matches

cancelExam / getExam	Admin / anyone	Cancel exam / read exam details

Events emitted: RoleGranted, ExamScheduled, PaperUploaded, PaperDownloaded, ReleaseApproved, KeyReleased, PaperVerified, LeakReported and ExamCancelled. Events are stored in transaction receipts and cannot be altered, so they form the audit trail.

5.2 Frontend dashboard

The dashboard (index.html, app.js, style.css) has tabs for Dashboard, Admin, Paper Setter, Authorities, Exam Center, Leak Check and Audit Log. An "Acting as" selector switches between the demo wallets so all roles can be demonstrated on one machine. Encryption and hashing are performed entirely in the browser using the Web Crypto API, so the plain question paper never leaves the setter's computer. If the contract rejects a transaction, its exact revert reason (for example "Exam has not started yet") is shown to the user.

5.3 Testing

Nine automated test cases written with Mocha/Chai cover role checks, one-time upload, early release, insufficient approvals, wrong key, double approval, successful release, download logging, verification and leak proof. Hardhat's time helpers are used to move the blockchain clock forward to the exam time.

6. Output

#	Test scenario	Expected result	Actual result
1	Non-admin grants a role / schedules exam	Transaction reverted	Pass
2	Non-setter uploads; setter uploads twice	Transaction reverted	Pass
3	Key release before exam time	"Exam has not started yet"	Pass
4	Key release with only one approval	"Not enough approvals"	Pass
5	Key release with a wrong key	"Key does not match commitment"	Pass
6	Double approval / outsider approval	Transaction reverted	Pass
7	Release at exam time with 2 of 3 approvals	Key published	Pass
8	Download logging and authenticity check	Events emitted	Pass
9	Leaked copy hash matches paper	Exam marked Compromised	Pass

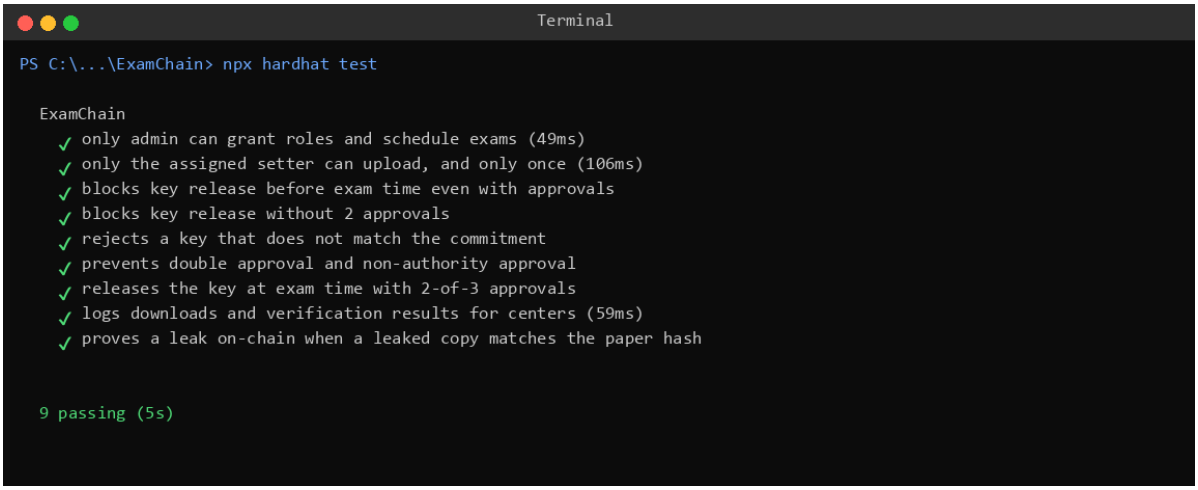


Figure 2: All 9 smart contract unit tests passing

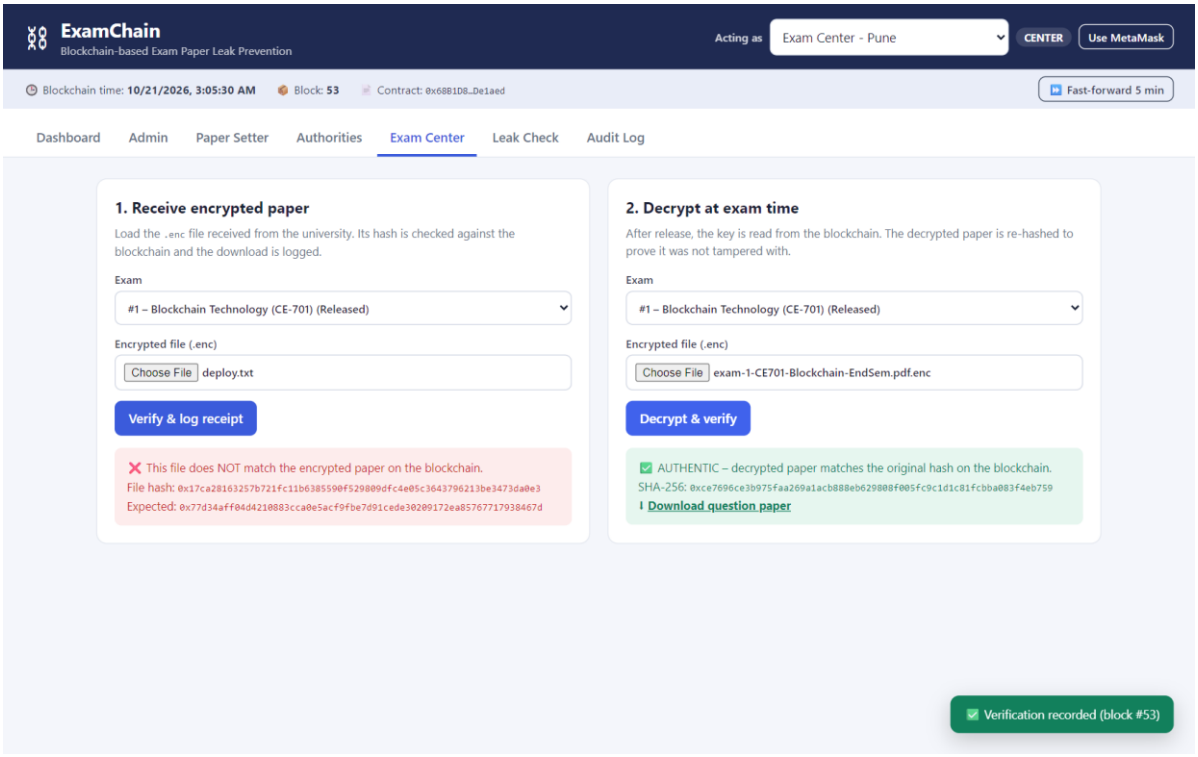


Figure 3: Exam center decrypts the paper at exam time – verified AUTHENTIC

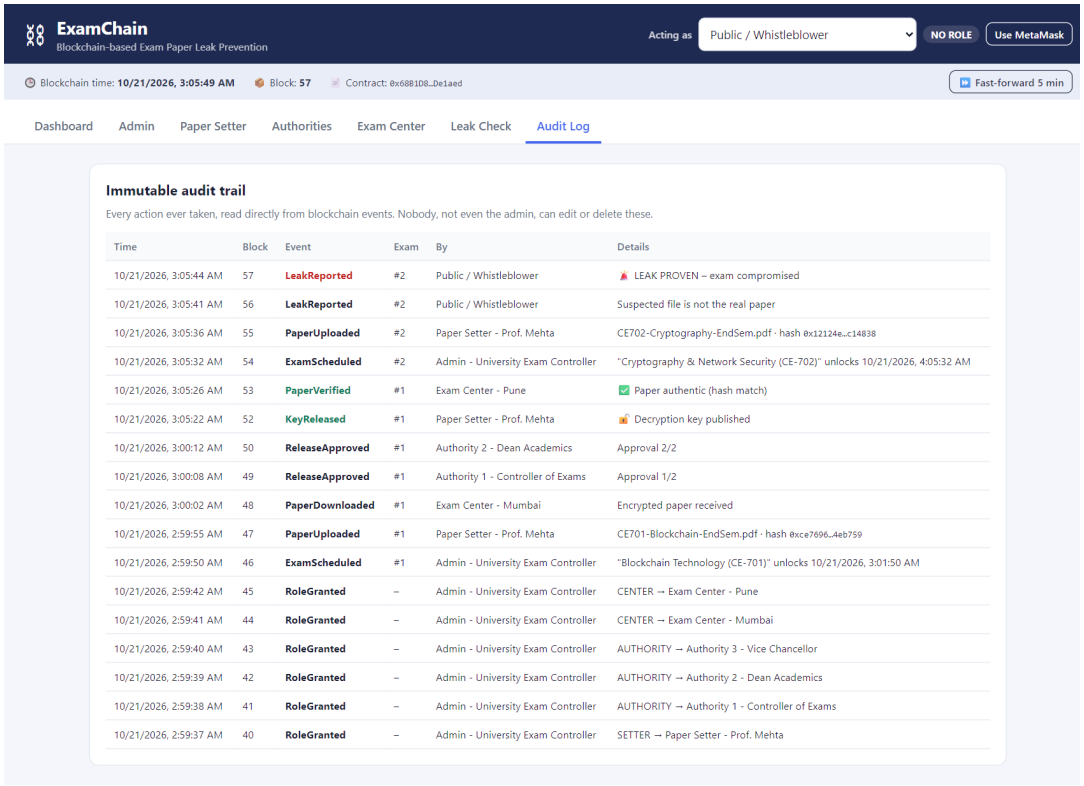


Figure 4: Immutable audit trail read from blockchain events

The complete step-by-step input and output screenshots are given in section B.2.

7. Conclusion

ExamChain demonstrates how blockchain immutability and smart-contract automation can remove blind trust from the examination process. The question paper remains encrypted until the exam starts, no single official can unlock it, every access is permanently recorded with a timestamp and wallet address, centers can prove the paper is genuine, and leaks can be proven cryptographically. The smart contract enforced every rule correctly in all test cases and in the live demonstration, making the examination system more secure, transparent and accountable.

REFERENCES

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[2] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper.

[3] Solidity Documentation, https://docs.soliditylang.org

[4] Hardhat Documentation, https://hardhat.org/docs

[5] ethers.js v6 Documentation, https://docs.ethers.org/v6

[6] NIST FIPS 197, "Advanced Encryption Standard (AES)"; NIST FIPS 180-4, "Secure Hash Standard (SHS)."

[7] I. Bashir, "Mastering Blockchain," Packt Publishing.

PART B

(PART B: TO BE COMPLETED BY STUDENTS)

Roll. No.:	Name:
Class: B.E. Computer Engineering (Sem VII)	Batch:
Date of Experiment:	Date of Submission: 21/10/2026
Grade:	

B.1 Software Code written by student:

(a) Smart contract – contracts/ExamChain.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title ExamChain - Blockchain-based exam paper leak prevention
/// @notice The question paper is encrypted off-chain (AES-256-GCM in the browser).
///         Only hashes and a commitment to the key are stored on-chain. The key can
///         be revealed only after the exam start time AND after 2-of-N exam
///         authorities approve. Every action is permanently logged as an event.
contract ExamChain {
    enum Status { Scheduled, Uploaded, Released, Compromised, Cancelled }

    struct Exam {
        string subject;
        address setter;
        uint256 unlockTime;     // exam start time (unix seconds)
        bytes32 paperHash;      // SHA-256 of the original (plain) paper
        bytes32 encryptedHash;  // SHA-256 of the encrypted .enc file
        bytes32 keyCommitment;  // keccak256 of the AES key
        bytes32 revealedKey;    // AES key, filled only after release
        string fileName;
        uint8 approvals;
        Status status;
    }

    uint8 public constant REQUIRED_APPROVALS = 2;

    address public admin;
    uint256 public examCount;

    mapping(address => bool) public isSetter;
    mapping(address => bool) public isAuthority;
    mapping(address => bool) public isCenter;

    mapping(uint256 => Exam) private exams;
    mapping(uint256 => mapping(address => bool)) public hasApproved;
    mapping(uint256 => mapping(address => bool)) public hasDownloaded;

    event RoleGranted(string role, address indexed account);
    event ExamScheduled(uint256 indexed examId, string subject, address indexed setter, uint256
unlockTime);
    event PaperUploaded(uint256 indexed examId, address indexed setter, bytes32 paperHash, bytes32
encryptedHash, string fileName);
    event PaperDownloaded(uint256 indexed examId, address indexed center, uint256 timestamp);
    event ReleaseApproved(uint256 indexed examId, address indexed authority, uint8 approvals);
    event KeyReleased(uint256 indexed examId, address indexed releasedBy, uint256 timestamp);
    event PaperVerified(uint256 indexed examId, address indexed center, bool authentic);
    event LeakReported(uint256 indexed examId, address indexed reporter, bool confirmed, uint256
timestamp);
    event ExamCancelled(uint256 indexed examId);

    modifier onlyAdmin() {
        require(msg.sender == admin, "Only admin");
        _;
    }
```

```
    modifier onlyCenter() {
        require(isCenter[msg.sender], "Only exam center");
        _;
    }

    modifier examExists(uint256 examId) {
        require(examId > 0 && examId <= examCount, "Exam does not exist");
        _;
    }

    constructor() {
        admin = msg.sender;
    }

    // ------------------------------------------------------------------ roles

    function addSetter(address account) external onlyAdmin {
        isSetter[account] = true;
        emit RoleGranted("SETTER", account);
    }

    function addAuthority(address account) external onlyAdmin {
        isAuthority[account] = true;
        emit RoleGranted("AUTHORITY", account);
    }

    function addCenter(address account) external onlyAdmin {
        isCenter[account] = true;
        emit RoleGranted("CENTER", account);
    }

    // ------------------------------------------------------------------ exam lifecycle

    function scheduleExam(string calldata subject, address setter, uint256 unlockTime)
        external
        onlyAdmin
        returns (uint256 examId)
    {
        require(isSetter[setter], "Not a registered setter");
        require(unlockTime > block.timestamp, "Unlock time must be in the future");

        examId = ++examCount;
        Exam storage e = exams[examId];
        e.subject = subject;
        e.setter = setter;
        e.unlockTime = unlockTime;
        e.status = Status.Scheduled;

        emit ExamScheduled(examId, subject, setter, unlockTime);
    }

    /// @notice Setter commits the encrypted paper. Once uploaded it can never be changed.
    function uploadPaper(
        uint256 examId,
        bytes32 paperHash,
        bytes32 encryptedHash,
        bytes32 keyCommitment,
        string calldata fileName
    ) external examExists(examId) {
        Exam storage e = exams[examId];
        require(msg.sender == e.setter, "Only assigned setter");
        require(e.status == Status.Scheduled, "Paper already uploaded");
        require(block.timestamp < e.unlockTime, "Exam already started");

        e.paperHash = paperHash;
        e.encryptedHash = encryptedHash;
        e.keyCommitment = keyCommitment;
        e.fileName = fileName;
        e.status = Status.Uploaded;

        emit PaperUploaded(examId, msg.sender, paperHash, encryptedHash, fileName);
    }

    /// @notice Centers log every download of the encrypted file (audit trail).
    function logDownload(uint256 examId) external onlyCenter examExists(examId) {
        Exam storage e = exams[examId];
        require(e.status == Status.Uploaded || e.status == Status.Released, "Paper not available");

        hasDownloaded[examId][msg.sender] = true;
```

```
        emit PaperDownloaded(examId, msg.sender, block.timestamp);
    }

    function approveRelease(uint256 examId) external examExists(examId) {
        require(isAuthority[msg.sender], "Only exam authority");
        Exam storage e = exams[examId];
        require(e.status == Status.Uploaded, "Paper not in uploaded state");
        require(!hasApproved[examId][msg.sender], "Already approved");

        hasApproved[examId][msg.sender] = true;
        e.approvals++;
        emit ReleaseApproved(examId, msg.sender, e.approvals);
    }

    /// @notice Time-locked, multi-approval key release. The key must match the
    ///         commitment made at upload time, so a fake key cannot be swapped in.
    function releaseKey(uint256 examId, bytes32 key) external examExists(examId) {
        Exam storage e = exams[examId];
        require(msg.sender == e.setter, "Only assigned setter");
        require(e.status == Status.Uploaded, "Paper not in uploaded state");
        require(block.timestamp >= e.unlockTime, "Exam has not started yet");
        require(e.approvals >= REQUIRED_APPROVALS, "Not enough approvals");
        require(keccak256(abi.encodePacked(key)) == e.keyCommitment, "Key does not match commitment");

        e.revealedKey = key;
        e.status = Status.Released;
        emit KeyReleased(examId, msg.sender, block.timestamp);
    }

    /// @notice Center records whether the decrypted paper matches the original hash.
    function recordVerification(uint256 examId, bytes32 computedHash) external onlyCenter
examExists(examId) {
        Exam storage e = exams[examId];
        require(e.status == Status.Released, "Key not released yet");

        emit PaperVerified(examId, msg.sender, computedHash == e.paperHash);
    }

    /// @notice Anyone who finds a suspected leaked copy before the exam can hash it and
    ///         report it. If the hash matches the committed paper, the leak is proven
    ///         on-chain and the exam is marked compromised.
    function reportLeak(uint256 examId, bytes32 suspectedHash) external examExists(examId) {
        Exam storage e = exams[examId];
        require(e.status == Status.Uploaded, "Can only report before release");

        bool confirmed = suspectedHash == e.paperHash;
        if (confirmed) {
            e.status = Status.Compromised;
        }
        emit LeakReported(examId, msg.sender, confirmed, block.timestamp);
    }

    function cancelExam(uint256 examId) external onlyAdmin examExists(examId) {
        Exam storage e = exams[examId];
        require(e.status != Status.Released, "Already released");

        e.status = Status.Cancelled;
        emit ExamCancelled(examId);
    }

    // ------------------------------------------------------------------ views

    function getExam(uint256 examId) external view examExists(examId) returns (Exam memory) {
        return exams[examId];
    }
}
```

(b) Deployment script – scripts/deploy.js

```
const fs = require("fs");
const path = require("path");
const { ethers, artifacts } = require("hardhat");

// Hardhat's default accounts are mapped to demo roles.
const ROLES = [
  { index: 0, role: "ADMIN", label: "Admin - University Exam Controller" },
  { index: 1, role: "SETTER", label: "Paper Setter - Prof. Mehta" },
  { index: 2, role: "AUTHORITY", label: "Authority 1 - Controller of Exams" },
  { index: 3, role: "AUTHORITY", label: "Authority 2 - Dean Academics" },
```

```
  { index: 4, role: "AUTHORITY", label: "Authority 3 - Vice Chancellor" },
  { index: 5, role: "CENTER", label: "Exam Center - Mumbai" },
  { index: 6, role: "CENTER", label: "Exam Center - Pune" },
  { index: 7, role: "PUBLIC", label: "Public / Whistleblower" },
];

async function main() {
  const signers = await ethers.getSigners();
  const chain = await ethers.deployContract("ExamChain");
  await chain.waitForDeployment();
  const address = await chain.getAddress();
  console.log("ExamChain deployed to:", address);

  const accounts = [];
  for (const r of ROLES) {
    const addr = signers[r.index].address;
    if (r.role === "SETTER") await (await chain.addSetter(addr)).wait();
    if (r.role === "AUTHORITY") await (await chain.addAuthority(addr)).wait();
    if (r.role === "CENTER") await (await chain.addCenter(addr)).wait();
    accounts.push({ ...r, address: addr });
    console.log(`  ${r.label.padEnd(40)} ${addr}`);
  }

  const { abi } = await artifacts.readArtifact("ExamChain");
  const config = { address, abi, accounts, rpcUrl: "http://127.0.0.1:8545" };
  const out = path.join(__dirname, "..", "frontend", "config.js");
  fs.writeFileSync(out, "window.EXAMCHAIN = " + JSON.stringify(config, null, 2) + ";\n");
  console.log("Frontend config written to frontend/config.js");
}

main().catch((err) => {
  console.error(err);
  process.exitCode = 1;
});
```

(c) Frontend – encryption and upload by paper setter (frontend/app.js)

```
  // Setter: encrypt in the browser, commit hashes on-chain
  $("#uploadForm").onsubmit = async (ev) => {
    ev.preventDefault();
    const f = ev.target;
    const id = f.exam.value;
    if (!id) return toast("Ask the admin to schedule an exam first", "err");
    const { file, bytes } = await readFile(f.file);

    const paperHash = await sha256(bytes);
    const keyBytes = crypto.getRandomValues(new Uint8Array(32));
    const iv = crypto.getRandomValues(new Uint8Array(12));
    const aesKey = await crypto.subtle.importKey("raw", keyBytes, "AES-GCM", false, ["encrypt"]);
    const cipher = new Uint8Array(await crypto.subtle.encrypt({ name: "AES-GCM", iv }, aesKey, bytes));
    const encrypted = new Uint8Array(iv.length + cipher.length);
    encrypted.set(iv);
    encrypted.set(cipher, iv.length);

    const encryptedHash = await sha256(encrypted);
    const keyHex = ethers.hexlify(keyBytes);
    const commitment = ethers.keccak256(keyBytes);

    const receipt = await send("uploadPaper", [id, paperHash, encryptedHash, commitment, file.name],
"Paper locked on blockchain");
    if (!receipt) return;

    localStorage.setItem("examchain-key-" + CFG.address + "-" + id, keyHex);
    result(
      "#uploadResult",
      "ok",
      `🔒 Paper encrypted and committed in block #${receipt.blockNumber}.<br>
       Paper SHA-256: <code>${paperHash}</code><br>
       Key commitment: <code>${commitment}</code><br><br>
       ⬇ ${blobLink(encrypted, `exam-${id}-${file.name}.enc`, "application/octet-stream", "Download
encrypted paper (.enc)")} – send this to exam centers.<br>
       ⬇ ${blobLink(keyHex, `exam-${id}-SECRET-KEY.txt`, "text/plain", "Download secret key")} – keep this
private until exam time.`
    );
  };
```

(d) Frontend – decryption and verification at exam center (frontend/app.js)

```
  // Center: decrypt with on-chain key and prove authenticity
  $("#decryptForm").onsubmit = async (ev) => {
    ev.preventDefault();
    const f = ev.target;
    const exam = examById(f.exam.value);
    if (!exam) return toast("No exam selected", "err");
    if (exam.status !== 2) {
      return result("#decryptResult", "info", `🔒 Key not released yet (status: ${STATUS[exam.status]}).
${countdown(exam.unlockTime)}. Approvals ${exam.approvals}/${REQUIRED_APPROVALS}.`);
    }
    const { bytes } = await readFile(f.file);
    let plain;
    try {
      const aesKey = await crypto.subtle.importKey("raw", ethers.getBytes(exam.revealedKey), "AES-GCM",
false, ["decrypt"]);
      plain = new Uint8Array(await crypto.subtle.decrypt({ name: "AES-GCM", iv: bytes.slice(0, 12) },
aesKey, bytes.slice(12)));
    } catch {
      return result("#decryptResult", "bad", "❌ Decryption failed – this is not the encrypted file for
this exam, or it was modified.");
    }
    const hash = await sha256(plain);
    const authentic = hash === exam.paperHash;
    const receipt = await send("recordVerification", [exam.id, hash], "Verification recorded");
    if (!receipt) return;

    const isPdf = exam.fileName.toLowerCase().endsWith(".pdf");
    const type = isPdf ? "application/pdf" : "application/octet-stream";
    const preview = isPdf ? `<iframe class="preview" src="${URL.createObjectURL(new Blob([plain], { type
}))}"></iframe>` : "";
    result(
      "#decryptResult",
      authentic ? "ok" : "bad",
      `${authentic ? "✅ AUTHENTIC – decrypted paper matches the original hash on the blockchain." : "❌
TAMPERED – hash does not match!"}<br>
       SHA-256: <code>${hash}</code><br>⬇ ${blobLink(plain, exam.fileName, type, "Download question
paper")}${preview}`
    );
  };
```

(e) Unit tests – test/ExamChain.test.js

```
const { expect } = require("chai");
const { ethers } = require("hardhat");
const { time } = require("@nomicfoundation/hardhat-network-helpers");

describe("ExamChain", function () {
  const KEY = ethers.hexlify(ethers.randomBytes(32));
  const COMMITMENT = ethers.keccak256(KEY);
  const PAPER_HASH = ethers.sha256(ethers.toUtf8Bytes("Q1. Explain proof of work."));
  const ENC_HASH = ethers.sha256(ethers.toUtf8Bytes("encrypted-bytes"));

  let chain, admin, setter, auth1, auth2, auth3, center, outsider, unlockTime;

  beforeEach(async function () {
    [admin, setter, auth1, auth2, auth3, center, outsider] = await ethers.getSigners();
    chain = await ethers.deployContract("ExamChain");

    await chain.addSetter(setter.address);
    await chain.addAuthority(auth1.address);
    await chain.addAuthority(auth2.address);
    await chain.addAuthority(auth3.address);
    await chain.addCenter(center.address);

    unlockTime = (await time.latest()) + 3600;
    await chain.scheduleExam("Blockchain Technology", setter.address, unlockTime);
  });

  async function upload() {
    await chain.connect(setter).uploadPaper(1, PAPER_HASH, ENC_HASH, COMMITMENT, "paper.pdf");
  }

  async function approveTwice() {
    await chain.connect(auth1).approveRelease(1);
    await chain.connect(auth2).approveRelease(1);
  }

  it("only admin can grant roles and schedule exams", async function () {
    await expect(chain.connect(outsider).addCenter(outsider.address)).to.be.revertedWith("Only admin");
```

```
    await expect(
      chain.connect(outsider).scheduleExam("X", setter.address, unlockTime)
    ).to.be.revertedWith("Only admin");
  });

  it("only the assigned setter can upload, and only once", async function () {
    await expect(
      chain.connect(outsider).uploadPaper(1, PAPER_HASH, ENC_HASH, COMMITMENT, "p.pdf")
    ).to.be.revertedWith("Only assigned setter");

    await upload();
    const exam = await chain.getExam(1);
    expect(exam.paperHash).to.equal(PAPER_HASH);
    expect(exam.status).to.equal(1); // Uploaded

    await expect(upload()).to.be.revertedWith("Paper already uploaded");
  });

  it("blocks key release before exam time even with approvals", async function () {
    await upload();
    await approveTwice();
    await expect(chain.connect(setter).releaseKey(1, KEY)).to.be.revertedWith("Exam has not started yet");
  });

  it("blocks key release without 2 approvals", async function () {
    await upload();
    await chain.connect(auth1).approveRelease(1);
    await time.increaseTo(unlockTime);
    await expect(chain.connect(setter).releaseKey(1, KEY)).to.be.revertedWith("Not enough approvals");
  });

  it("rejects a key that does not match the commitment", async function () {
    await upload();
    await approveTwice();
    await time.increaseTo(unlockTime);
    const fakeKey = ethers.hexlify(ethers.randomBytes(32));
    await expect(chain.connect(setter).releaseKey(1, fakeKey)).to.be.revertedWith(
      "Key does not match commitment"
    );
  });

  it("prevents double approval and non-authority approval", async function () {
    await upload();
    await chain.connect(auth1).approveRelease(1);
    await expect(chain.connect(auth1).approveRelease(1)).to.be.revertedWith("Already approved");
    await expect(chain.connect(outsider).approveRelease(1)).to.be.revertedWith("Only exam authority");
  });

  it("releases the key at exam time with 2-of-3 approvals", async function () {
    await upload();
    await approveTwice();
    await time.increaseTo(unlockTime);
    await expect(chain.connect(setter).releaseKey(1, KEY)).to.emit(chain, "KeyReleased");

    const exam = await chain.getExam(1);
    expect(exam.revealedKey).to.equal(KEY);
    expect(exam.status).to.equal(2); // Released
  });

  it("logs downloads and verification results for centers", async function () {
    await upload();
    await expect(chain.connect(outsider).logDownload(1)).to.be.revertedWith("Only exam center");
    await expect(chain.connect(center).logDownload(1)).to.emit(chain, "PaperDownloaded");
    expect(await chain.hasDownloaded(1, center.address)).to.equal(true);

    await approveTwice();
    await time.increaseTo(unlockTime);
    await chain.connect(setter).releaseKey(1, KEY);

    await expect(chain.connect(center).recordVerification(1, PAPER_HASH))
      .to.emit(chain, "PaperVerified")
      .withArgs(1, center.address, true);
    await expect(chain.connect(center).recordVerification(1, ENC_HASH))
      .to.emit(chain, "PaperVerified")
      .withArgs(1, center.address, false);
  });

  it("proves a leak on-chain when a leaked copy matches the paper hash", async function () {
    await upload();
```

```
    await chain.connect(outsider).reportLeak(1, ENC_HASH);
    expect((await chain.getExam(1)).status).to.equal(1); // still Uploaded

    await expect(chain.connect(outsider).reportLeak(1, PAPER_HASH))
      .to.emit(chain, "LeakReported");
    expect((await chain.getExam(1)).status).to.equal(3); // Compromised

    await expect(chain.connect(auth1).approveRelease(1)).to.be.revertedWith("Paper not in uploaded
state");
    await time.increaseTo(unlockTime);
    await expect(chain.connect(setter).releaseKey(1, KEY)).to.be.revertedWith("Paper not in uploaded
state");
  });
});
```

B.2 Input and Output:

Step 1: Compile, test and deploy

Input: npx hardhat test, then npm run deploy on the local Hardhat network.

Output: 9 tests pass. The contract is deployed and 8 wallets receive their roles.

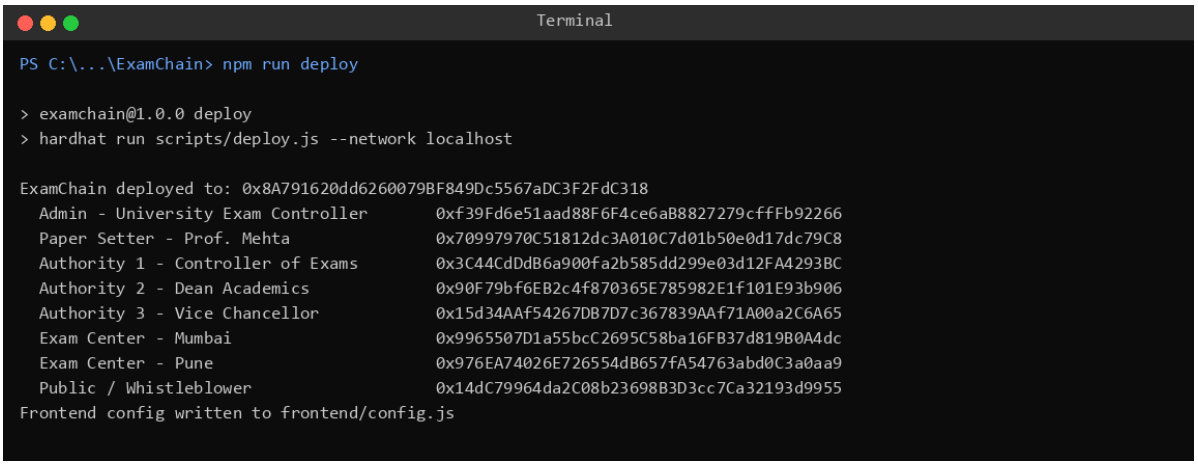


Figure 5: Contract deployment and role assignment

Step 2: Dashboard

Input: Open http://localhost:3000 as the Admin.

Output: Dashboard shows blockchain time, current block, contract address and the role of the active wallet.

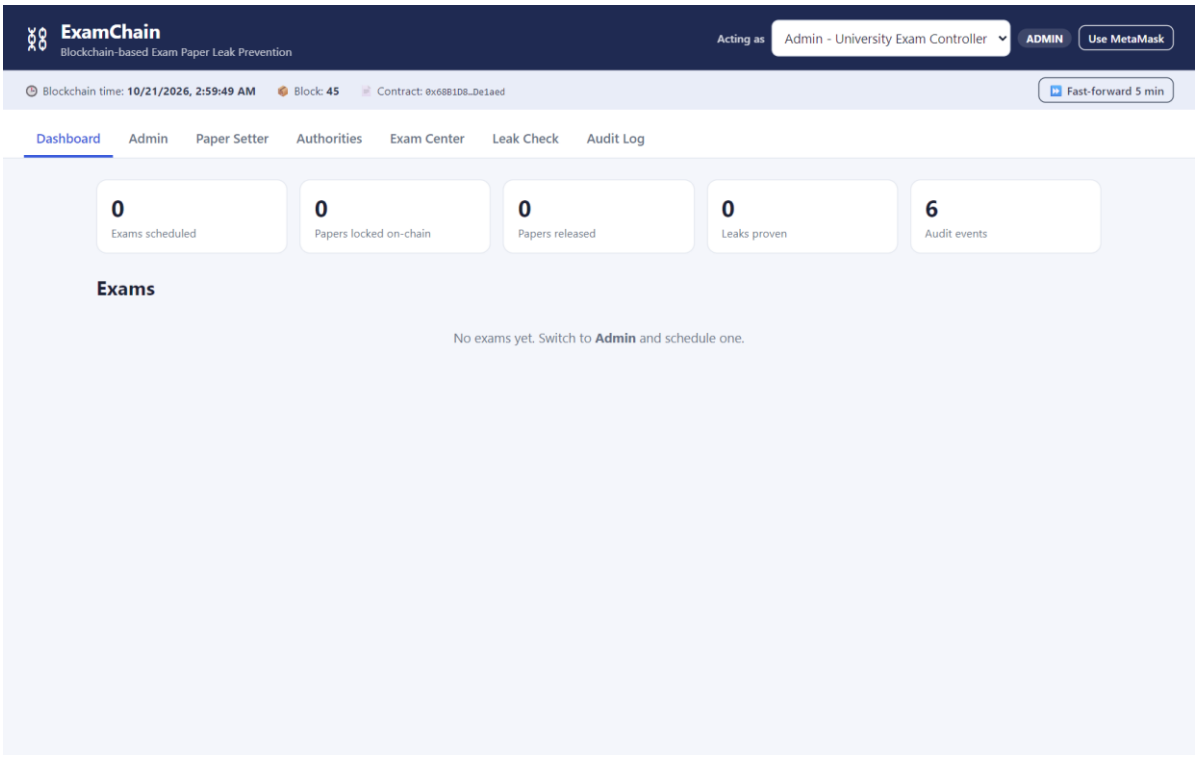


Figure 6: ExamChain dashboard before any exam is scheduled

Step 3: Admin schedules an exam

Input: Subject "Blockchain Technology (CE-701)", setter Prof. Mehta, unlock time = now + 2 minutes.

Output: ExamScheduled transaction confirmed in a new block.

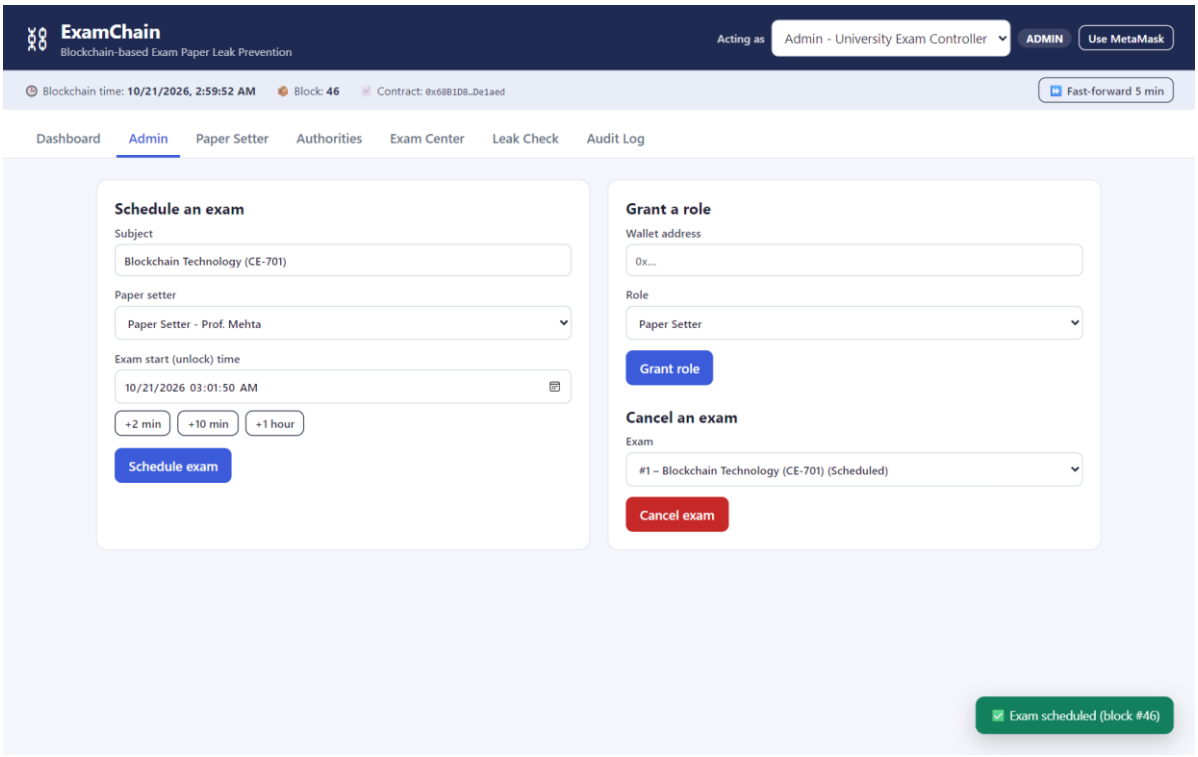


Figure 7: Admin schedules an exam with a time-lock

Step 4: Paper setter encrypts and uploads the paper

Input: Question paper CE701-Blockchain-EndSem.pdf selected by the Paper Setter.

Output: Paper encrypted in the browser. Its SHA-256 hash and key commitment are stored on-chain; the .enc file and the secret key are offered for download.

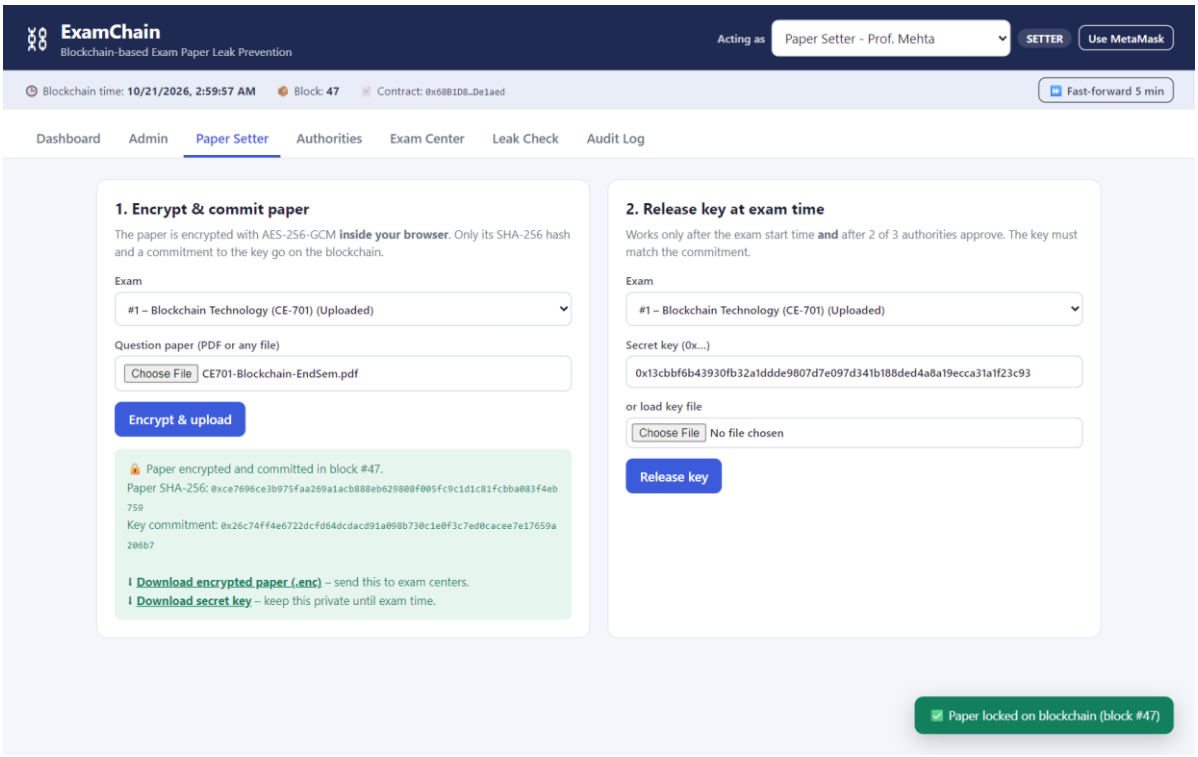


Figure 8: Paper encrypted with AES-256-GCM and committed on the blockchain

Step 5: Attempt to release the key before the exam

Input: Setter clicks Release key before the unlock time.

Output: Smart contract rejects the transaction: "Exam has not started yet".

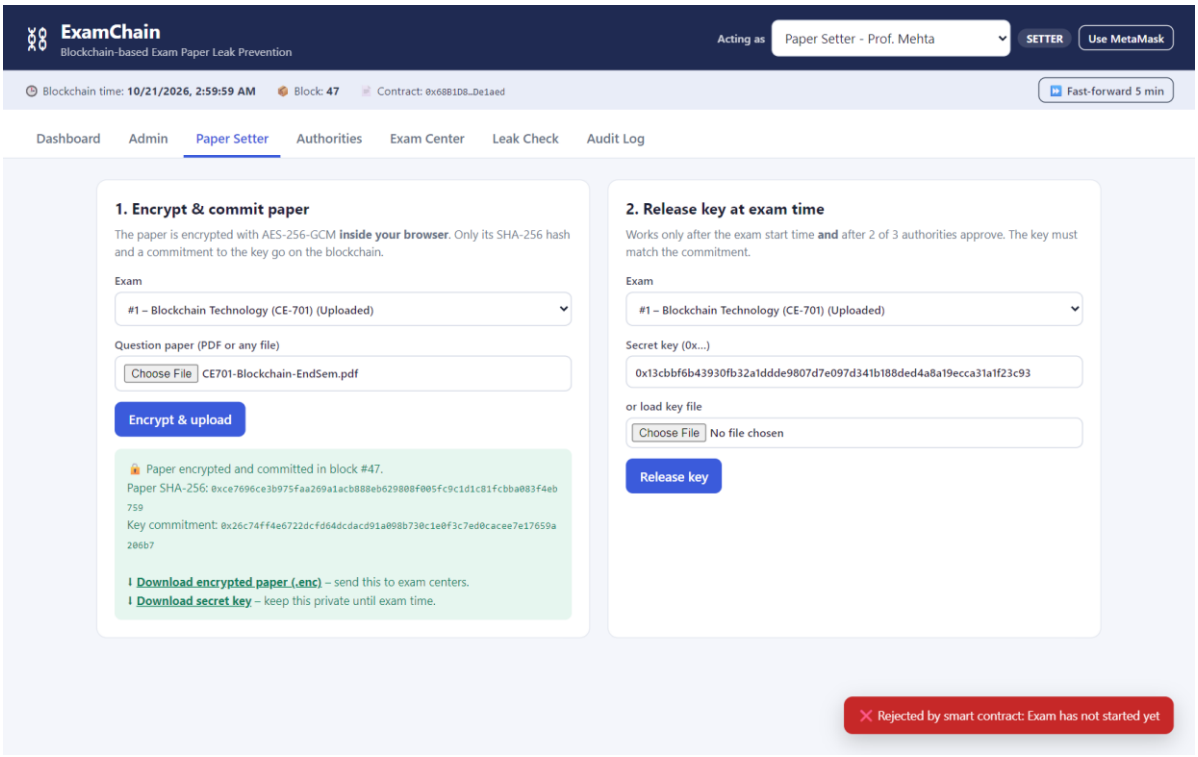


Figure 9: Time-lock prevents early release of the key

Step 6: Exam center receives the encrypted paper

Input: Exam Center – Mumbai loads the .enc file.

Output: Hash matches the on-chain encryptedHash; the receipt is logged on the blockchain.

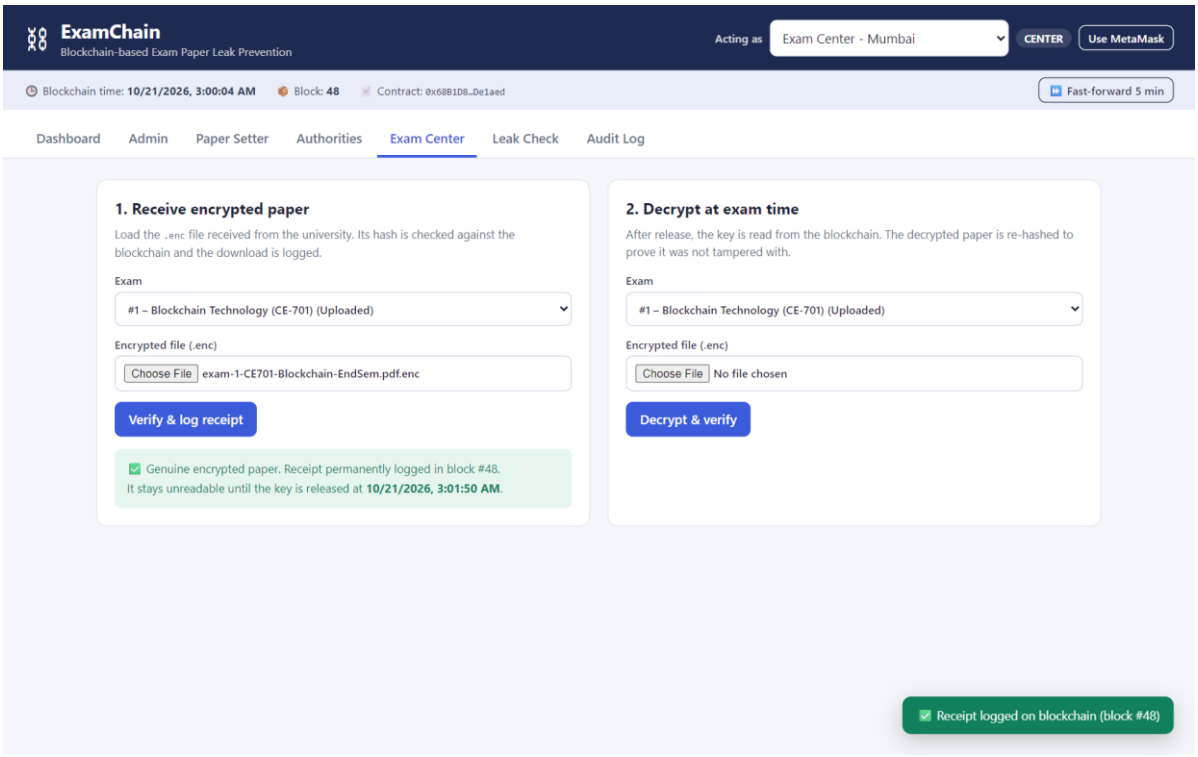


Figure 10: Genuine encrypted paper verified and receipt logged

Step 7: Tampered / wrong file

Input: A different file is loaded instead of the genuine .enc file.

Output: Hash mismatch is detected and shown with the expected hash; nothing is logged.

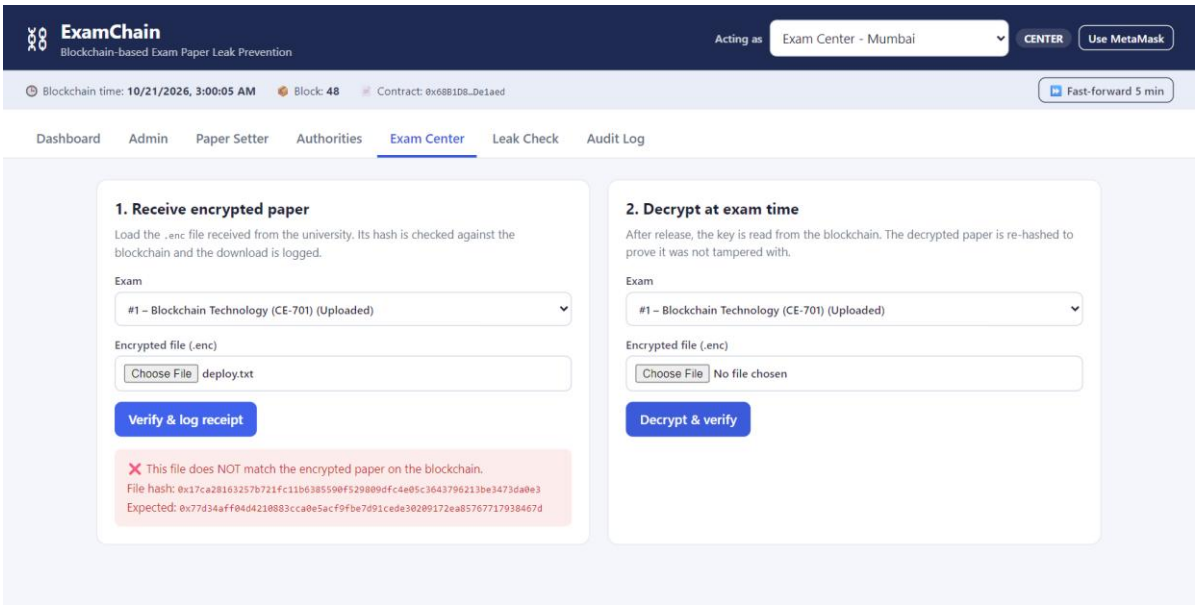


Figure 11: Tampered file detected using SHA-256

Step 8: Authorities approve the release

Input: Authority 1 (Controller of Exams) and Authority 2 (Dean Academics) click Approve release.

Output: Approvals 1/2 and then 2/2 are recorded on-chain.

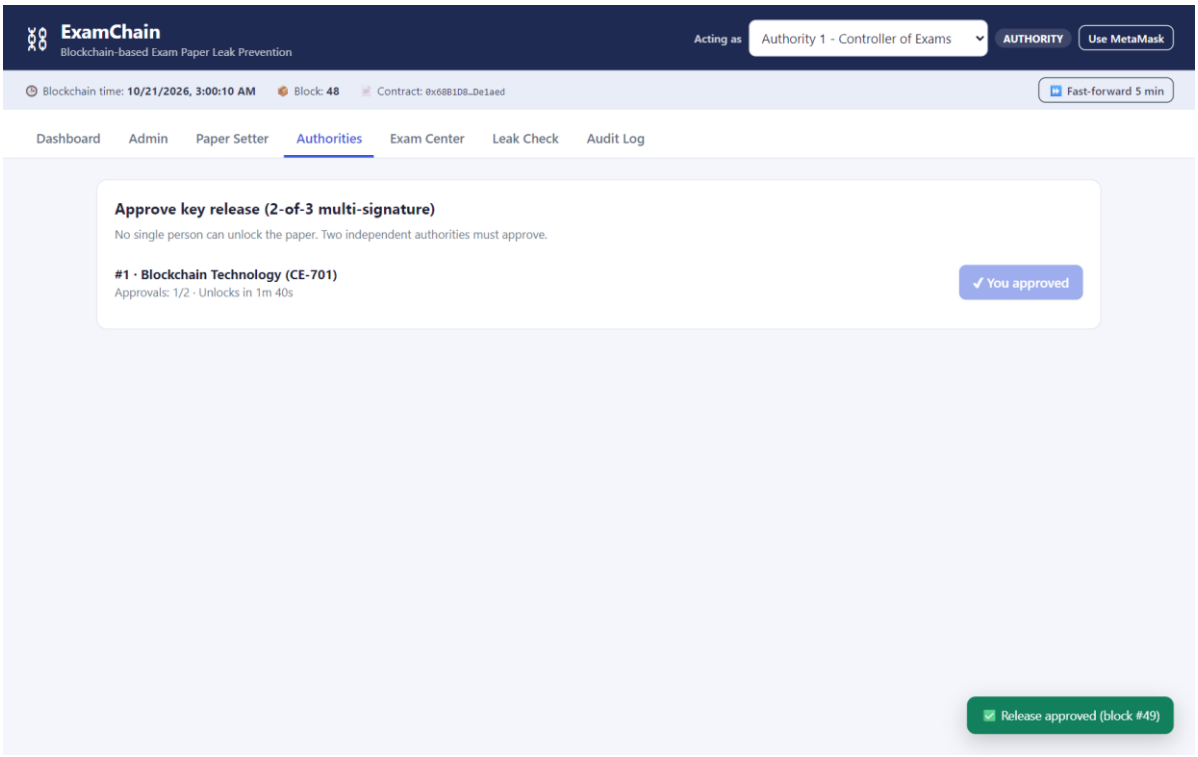


Figure 12: Multi-authority approval (2 of 3)

Step 9: Unauthorized user

Input: A wallet without any role (Public) tries to approve the release.

Output: Smart contract rejects it: "Only exam authority".

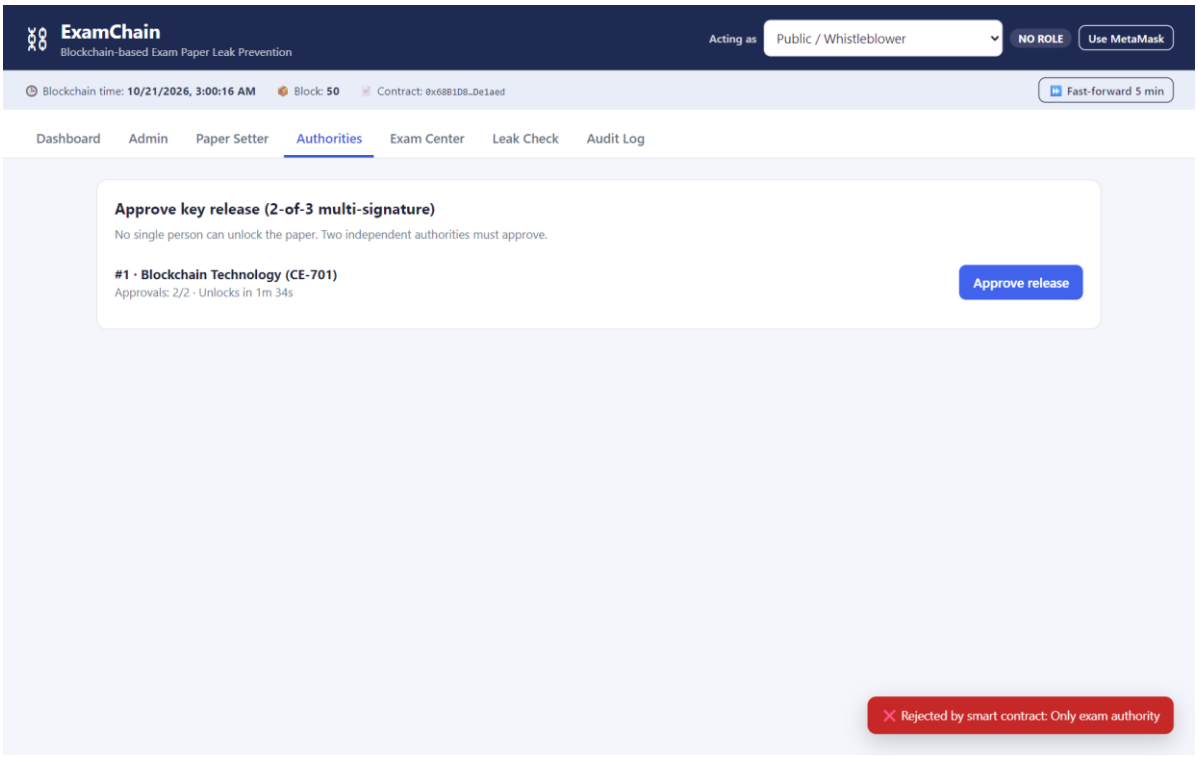


Figure 13: Role-based access control enforced by the smart contract

Step 10: Key release at exam time

Input: After the unlock time (clock advanced with Fast-forward for the demo), the setter submits the key.

Output: Time, approvals and commitment checks pass; the key is published and the exam status becomes Released.

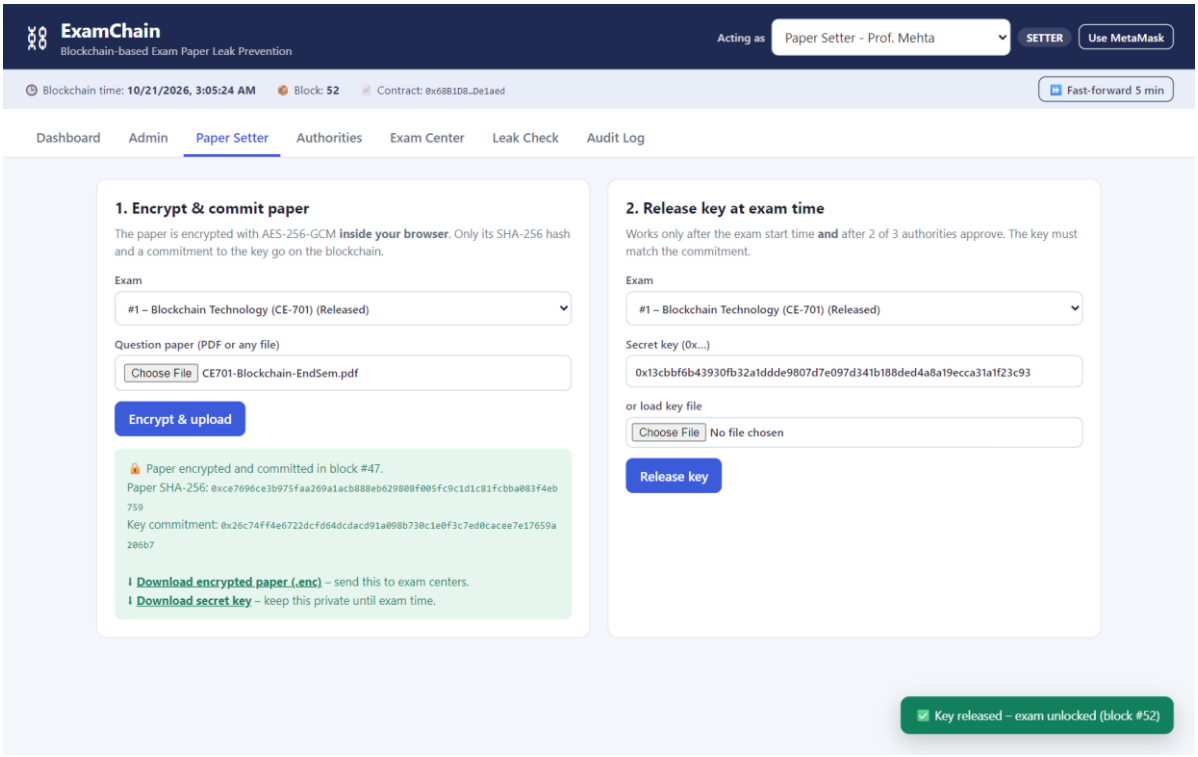


Figure 14: Key released after the time-lock and approvals

Step 11: Exam center decrypts and verifies

Input: Exam Center – Pune loads the .enc file and clicks Decrypt & verify.

Output: Key is read from the blockchain, the paper is decrypted, and its hash matches the original: AUTHENTIC.

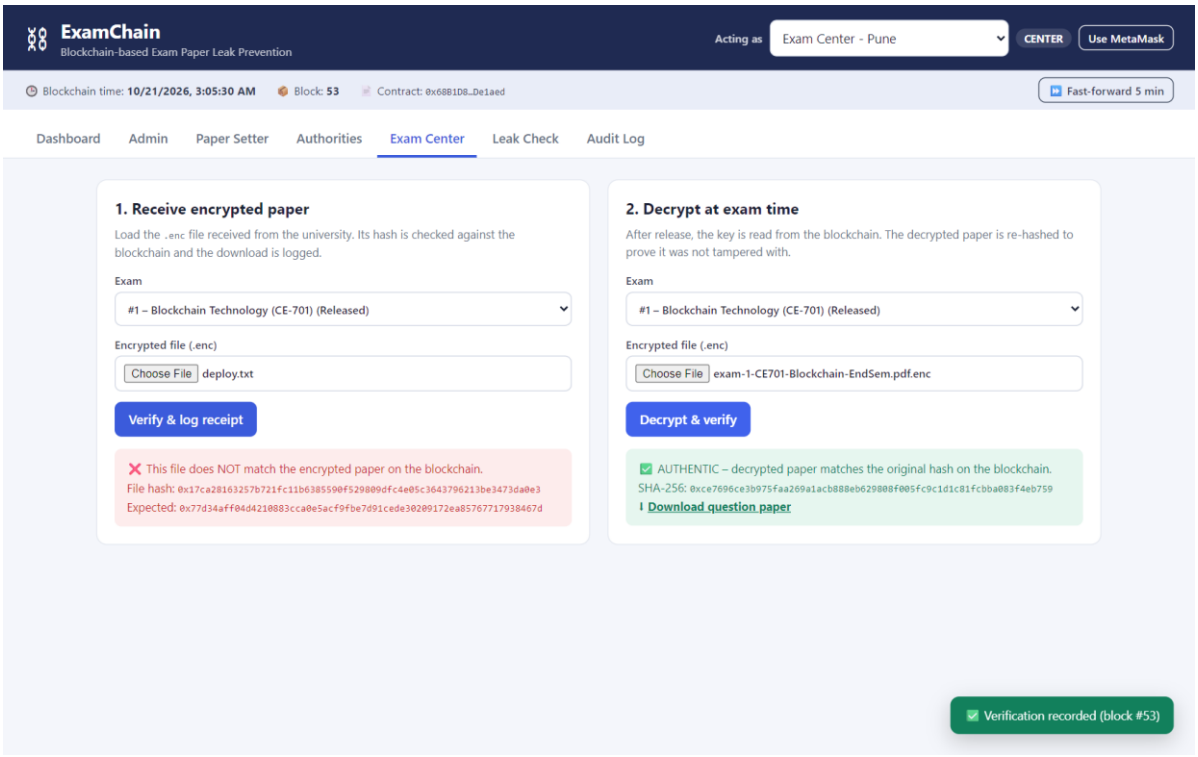


Figure 15: Decryption and authenticity verification

UNIVERSITY END SEMESTER EXAMINATION

B.E. Computer Engineering - Semester VII

CE-701 Blockchain Technology Max Marks: 80 Time: 3 Hours

Q1. (a) Explain the structure of a block and how blocks are linked using hashes. [10]

(b) Compare Proof of Work and Proof of Stake consensus mechanisms. [10]

Q2. (a) What is a Merkle tree? Explain its role in SPV. [10]

(b) Write a Solidity smart contract for a simple voting system. [10]

Q3. (a) Explain the Ethereum Virtual Machine and gas. [10]

(b) Discuss the double-spending problem and how Bitcoin solves it. [10]

Q4. (a) Explain Hyperledger Fabric architecture. [10]

(b) Describe any two real-world applications of blockchain. [10]

Figure 16: Decrypted question paper (byte-identical to the original, as proven by the hash match)

Step 12: Leak check with a fake file

Input: A second exam (CE-702) is scheduled and uploaded. A whistleblower submits an unrelated file as a suspected leak.

Output: Hashes do not match, so the file is reported as not the real paper. The report is still logged.

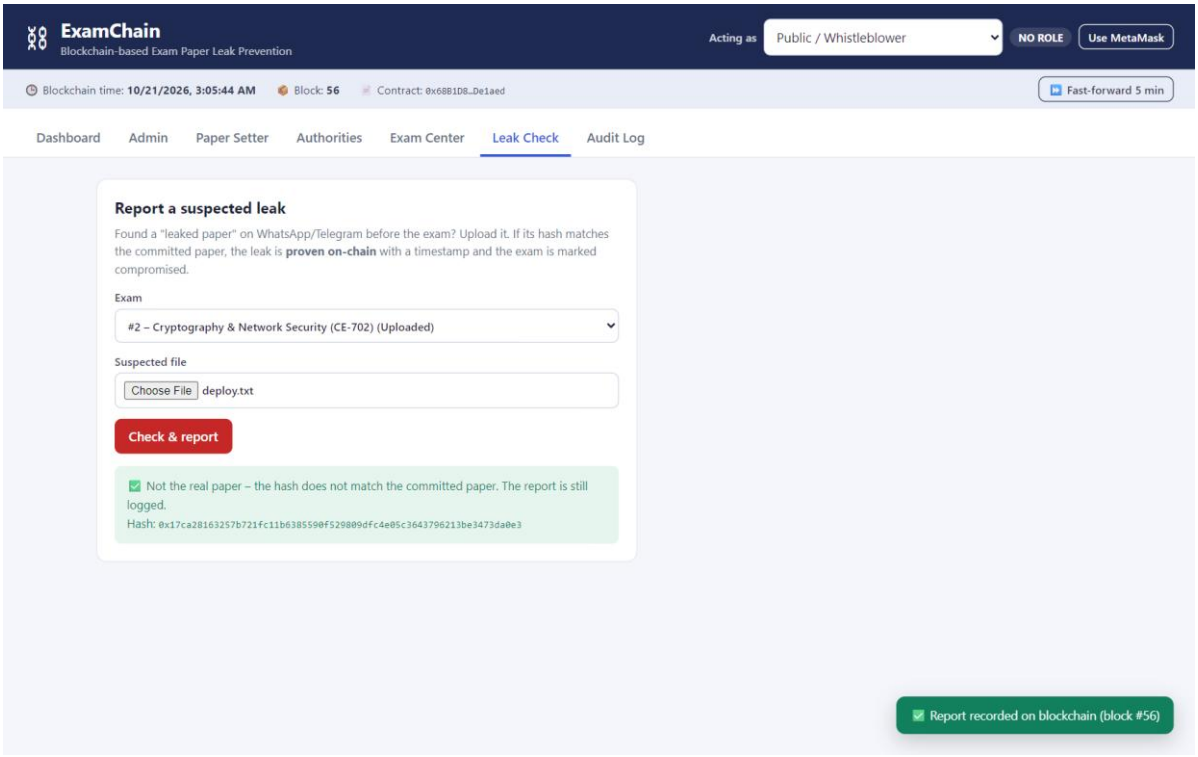


Figure 17: Fake leaked paper identified

Step 13: Leak proven

Input: The whistleblower submits an exact copy of the CE-702 paper before the exam.

Output: Hash matches the committed paper: LEAK PROVEN on-chain and the exam is marked Compromised.

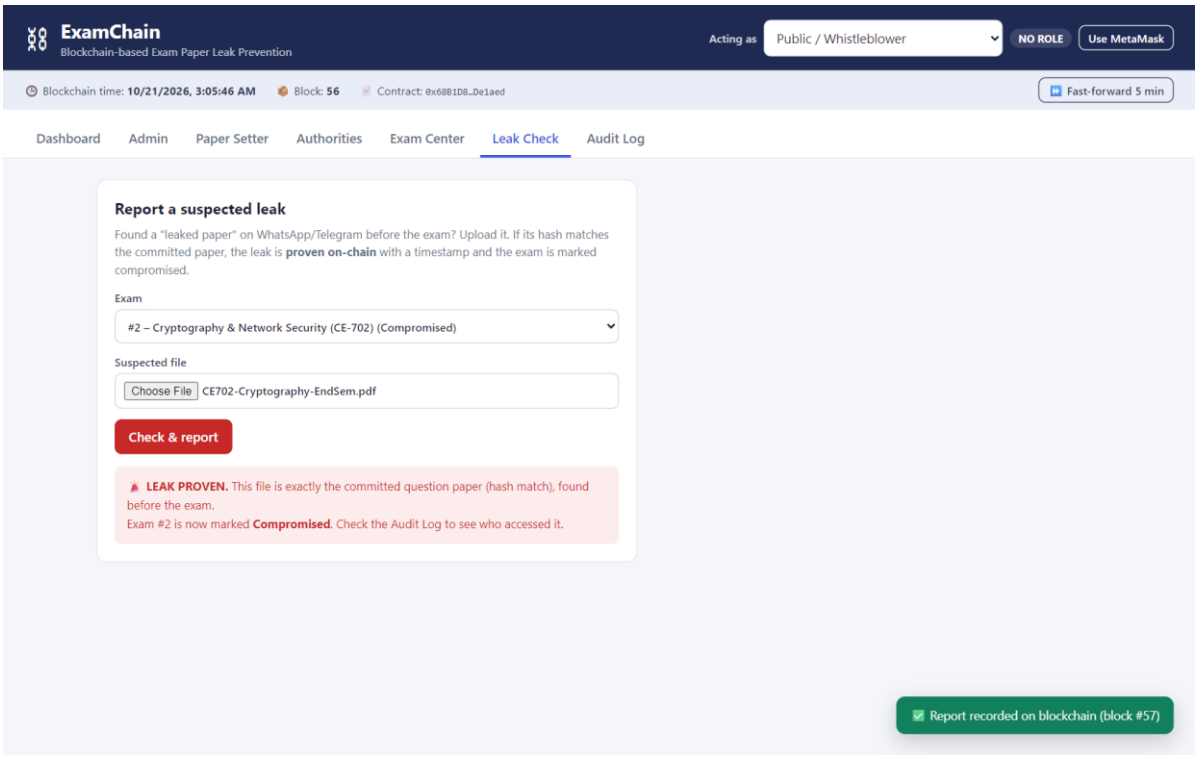


Figure 18: Leak proven cryptographically on the blockchain

Step 14: Final dashboard and audit log

Input: Open the Dashboard and Audit Log tabs.

Output: Exam #1 Released with 2/2 approvals; exam #2 Compromised. Every action is listed with time, block number and wallet.

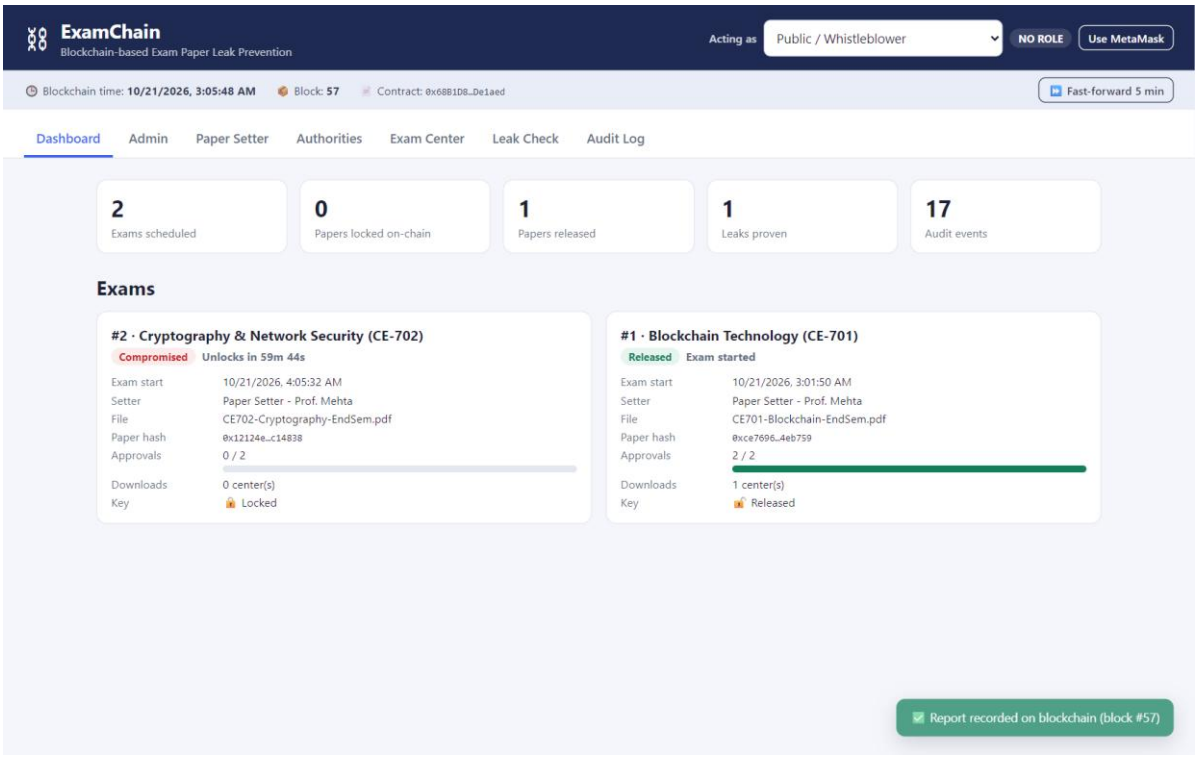


Figure 19: Final dashboard showing exam status

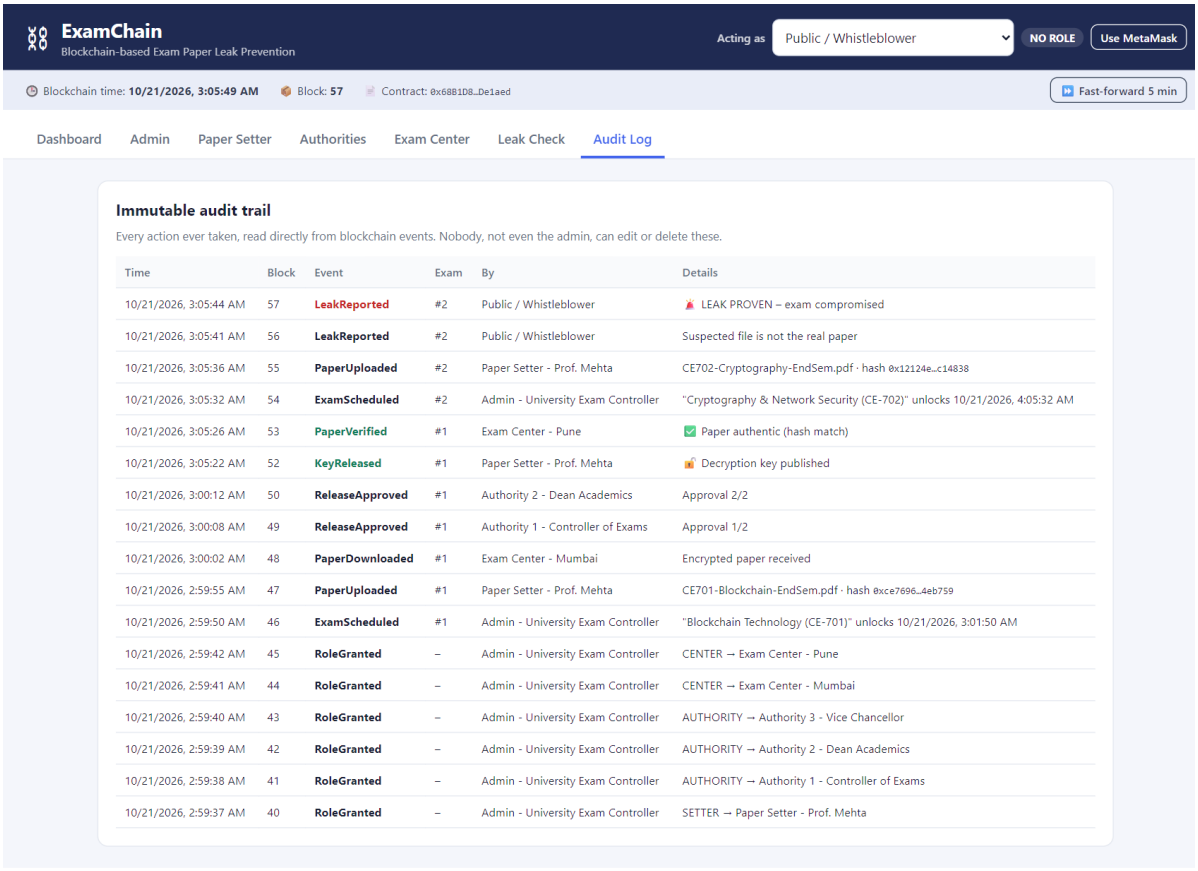


Figure 20: Complete audit trail of the demonstration

B.3 Observations and learning:

- **Immutability:** Once the paper hash and key commitment were stored, the setter could not upload again ("Paper already uploaded"), so the paper cannot be silently replaced.
- **Time-lock:** The contract rejected key release before the unlock time even when both approvals were present, showing that block.timestamp can enforce time-based rules without trusting any person.
- **Multi-authority control:** Release failed with one approval and succeeded only after two, removing the single point of trust. A wallet without a role was always rejected.
- **Commit-reveal:** A random key that did not match the Keccak-256 commitment was rejected, so the paper cannot be swapped at release time.
- **Integrity using hashes:** Changing even one byte of a file completely changed its SHA-256 hash, which allowed detection of tampered files and proof of leaks.
- **On-chain vs off-chain storage:** Storing only 32-byte hashes on-chain keeps gas cost low and keeps the paper confidential, since all blockchain data is publicly readable.
- **Events as audit logs:** Solidity events provided a cheap, searchable and permanent record of who did what and when, identified by wallet address and block number.
- **Learning:** Gained practical experience in writing and testing Solidity contracts, using Hardhat, connecting a web frontend with ethers.js, and combining symmetric encryption and hashing with blockchain.

B.4 Conclusion:

The mini project successfully implemented a blockchain-based exam paper leak prevention system. All objectives were achieved: the paper stayed encrypted until the exam time, the key could only be released after 2-of-3 authority approvals, every action was recorded in an immutable audit trail, exam centers verified authenticity using hashes, and a leaked copy was proven on-chain. All 9 unit tests passed and the complete workflow worked in the live demonstration. The project shows that blockchain is well suited to processes that need transparency, accountability and tamper-proof records without relying on a single trusted authority. As future scope, threshold encryption or Shamir's Secret Sharing can make the key release fully automatic, IPFS can store encrypted papers, and the system can be deployed on a permissioned network such as Hyperledger Besu.

B.5 Question of Curiosity

Q1. Why is the question paper not stored directly on the blockchain?

Everything stored on a public blockchain is readable by anyone and storage is very expensive in gas. Therefore only the 32-byte SHA-256 hashes and the key commitment are stored on-chain, while the encrypted file is distributed off-chain. The hashes are enough to verify integrity.

Q2. How does the smart contract implement the time-lock?

releaseKey() contains require(block.timestamp >= unlockTime). Validators set block.timestamp, so any transaction attempting to release the key before the exam start time is reverted.

Q3. What is a commit-reveal scheme and why is it used here?

At upload time the setter commits to Keccak-256(key) without revealing the key. At exam time the key is revealed and the contract re-hashes it and compares it with the commitment. This guarantees the released key is the same one used to encrypt the paper, so a wrong or fake key cannot be published.

Q4. Can the paper setter still leak the paper? How can this be improved?

The setter writes the paper, so they could leak it outside the system. ExamChain makes such a leak provable and traceable through the audit trail. It can be improved with threshold cryptography (the key is split among authorities using Shamir's Secret Sharing) or time-lock encryption (e.g. drand/tlock), so no single person ever holds the complete key.

Q5. Would a public or a permissioned blockchain be better for a university?

A permissioned blockchain (e.g. Hyperledger Besu or Fabric) suits a university better: participants are known, there are no gas fees, and transactions are faster. A public chain gives stronger public verifiability. The ExamChain contract logic remains the same in both cases.